

Question 1 : What is the difference between AI, ML, DL, and Data Science? Provide a brief explanation of each?

Reasons for Higher Ratings in Categories like Education

Education apps often achieve higher average ratings (typically 4.3–4.6) due to their focused, goal-oriented design that aligns with user needs. Interactive features like quizzes, progress tracking, visual animations, and systematic learning paths keep users engaged without stress, leading to positive feedback. Motivated, long-term users (students/parents) value clear benefits like performance analysis and reminders, resulting in higher satisfaction and ratings.

Reasons for Lower Ratings in Categories like Games

Games and similar categories (e.g., Social, Tools) average lower ratings (3.8–4.1) because users only review when extremely frustrated—bugs, crashes, or misleading ads prompt 1-star ratings, while satisfied users rarely rate. High competition means harsh comparisons, with complaints about paywalls, battery drain, and "pay-to-win" mechanics pulling averages down. Technical demands across devices amplify issues like poor performance, further hurting scores.

Other Influencing Factors

Monetization style matters: Fewer intrusive ads in utility-focused categories boost ratings, unlike ad-heavy games. Category competition affects scrutiny—niche categories like Education face less direct rivalry than broad ones like Games. Regular updates and strong retention signal quality to users, sustaining higher averages in stable categories.

What is the difference between AI, ML, DL, and Data Science? Provide a brief explanation of each.

Here is a brief explanation of the differences between AI, ML, DL, and Data Science:

- Artificial Intelligence (AI): AI is the broad field of computer science focused on creating machines that can simulate human intelligence, including reasoning, learning, and decision-making. It involves algorithms that enable machines to perform tasks that typically require human intelligence.
- Machine Learning (ML): ML is a subset of AI where computers learn from data and improve their performance over time without explicit programming. It uses statistical methods to find patterns in data and make predictions or decisions based on that data.
- Deep Learning (DL): DL is a further subset of ML that uses layered neural networks, inspired by the human brain's structure, to automatically learn features and perform complex tasks like image and speech recognition. It requires large amounts of data and computing power.
- Data Science: Data Science involves collecting, processing, analyzing, and interpreting large volumes of data to extract meaningful insights and support decision-making. It uses techniques from statistics, ML, and AI to uncover patterns and answer questions from data.

In summary, AI is the broad concept of intelligent machines, ML is a way to achieve AI through learning from data, DL is a specialized form of ML using neural networks, and Data Science is the discipline of extracting knowledge from data using these and other techniques

Here is a concise comparison of AI, ML, DL, and Data Science based on scope, techniques, and applications:

Aspect	Artificial Intelligence (AI)	Machine Learning (ML)	Deep Learning (DL)	Data Science

Scope	Broad field aiming to create intelligent machines	Subset of AI focused on learning from data	Subset of ML using multi-layer neural networks	Interdisciplinary field to extract insights from data
Techniques	Rule-based systems, expert systems, search algorithms	Statistical models like regression, classification	Neural networks (CNNs, RNNs), feature learning	Statistics, ML, data mining, visualization
Data Dependency	Can work with pre-defined rules or data	Requires labeled data to train models	Needs large datasets and high computational power	Works with structured/unstructured data
Applications	Robotics, chatbots, autonomous vehicles	Fraud detection, recommendations, spam filtering	Image/speech recognition, NLP, autonomous driving	Business intelligence, predictive analytics, healthcare insights

Summary:

AI is the overarching concept of intelligent machines. ML is a way to achieve AI via learning patterns from data. DL is a specialized ML form that uses neural networks for complex tasks. Data Science combines these techniques with statistics and domain knowledge to analyze data and extract actionable insights.

Q.2.Explain overfitting and underfitting in ML. How can you detect and prevent them?

Hint: Discuss bias-variance tradeoff, cross-validation, and regularization techniques

Overfitting and underfitting are common problems in machine learning related to how well a model generalizes to new data.

- Overfitting: Occurs when a model learns the training data too well, including noise and outliers. It has low bias but high variance, meaning it fits training data perfectly but performs poorly on unseen data. It memorizes rather than generalizes.
- Underfitting: Happens when a model is too simple to capture the underlying pattern of the data. It has high bias but low variance, leading to poor performance on both training and test data as it misses important relationships.

Detecting Overfitting and Underfitting

- Overfitting: High accuracy on training data but low accuracy on validation/test data. The model shows large performance gap between train and test sets.
- Underfitting: Poor performance on both training and test sets, indicating the model is too simple.

Bias-Variance Tradeoff

- Bias is error from overly simplistic assumptions (underfitting).
- Variance is error from too much complexity and sensitivity to training data (overfitting).
- The tradeoff is to balance bias and variance to minimize total error and optimize generalization.

Prevention Techniques

- Cross-Validation: Evaluate model performance on multiple data splits to detect overfitting early and tune hyperparameters accordingly.
- Regularization: Methods like L1 (Lasso) and L2 (Ridge) add a penalty for model complexity during training to reduce overfitting by controlling weights.
- Simplifying the model: Use fewer features or simpler algorithms if underfitting.
- Increasing training data: More data can help reduce overfitting by allowing better generalization.
- Early stopping: Stop training when validation error starts to increase.
- Ensemble Methods: Combine multiple models (bagging, boosting) to reduce variance.

In summary, managing overfitting and underfitting requires understanding the bias-variance tradeoff and applying techniques like cross-validation and regularization to build models that generalize well on unseen data.

Question 3:How would you handle missing values in a dataset? Explain at least three methods with examples. Hint: Consider deletion, mean/median imputation, and predictive modeling.

Missing values in datasets are handled using deletion, mean/median imputation, and predictive modeling to ensure reliable analysis and modeling.

Deletion Method

Rows or columns containing missing values are removed using functions like pandas' `dropna()`. For a dataset with Age and Salary:

text

Age	Salary
-----	--------

```
0  25.0  50000.0
1   NaN  60000.0
2  30.0   NaN
```

After row deletion (`dropna(how='any')`), only complete rows remain:

text

```
      Age  Salary
0  25.0  50000.0
3  35.0  70000.0
6  28.0  65000.0
```

This method works well for low missing rates (<5%) but can bias results if data is not missing completely at random (MCAR).

Mean/Median Imputation

Missing values are filled with the column's mean (average) or median (middle value). In the Age column (mean=31.6, median=30.0):

text

Mean: [25.0, 31.6, 30.0, 35.0, 31.6, 40.0, 28.0]

Median: [25.0, 30.0, 30.0, 35.0, 30.0, 40.0, 28.0]

Use `df['Age'].fillna(df['Age'].mean())`; median is preferable for skewed data or outliers as it reduces variance distortion.

Predictive Modeling

Advanced techniques like KNN Imputer or Multiple Imputation by Chained Equations (MICE) predict values based on other features. KNN (n_neighbors=2) on the dataset yields:

text

	Age	Salary
0	25.0	50000.0
1	31.5	60000.0
2	30.0	67500.0
4	26.5	55000.0

Question 4:What is an imbalanced dataset? Describe two techniques to handle it (theoretical + practical). Hint: Discuss SMOTE, Random Under/Oversampling, and class weights in models.

An imbalanced dataset is one where the number of samples across different classes is significantly uneven, causing a model to be biased toward the majority class and perform poorly on the minority class.

Two common techniques to handle imbalanced datasets are:

1. SMOTE (Synthetic Minority Over-sampling Technique):
 - Theoretical: SMOTE creates synthetic data points for the minority class by interpolating between existing minority samples and their nearest

neighbors. This avoids simply duplicating samples and reduces overfitting.

- Practical: Using libraries like imbalanced-learn in Python, you can apply SMOTE to generate new minority class samples, balancing the dataset before training.

2. Class Weights in Models:

- Theoretical: Class weights assign higher importance to minority class errors during model training by adjusting the loss function. This way, the model is penalized more for misclassifying minority class samples, improving its attention to them.
- Practical: In scikit-learn, many classifiers support a `class_weight` parameter (e.g., `class_weight='balanced'`) to automatically adjust weights based on class frequencies without changing the dataset.

Another related method is Random Over-sampling or Under-sampling:

- Random Over-sampling duplicates minority class samples to balance the data, but risks overfitting.
- Random Under-sampling reduces majority class samples, risking loss of information.

SMOTE generally improves over random sampling by synthesizing new data rather than duplicating existing points.

Question 5: Why is feature scaling important in ML? Compare Min-Max scaling and Standardization. Hint: Explain impact on distance-based algorithms (e.g., KNN, SVM) and gradient descent.

Question 5: Why is feature scaling important in ML? Compare Min-Max scaling and Standardization. Hint: Explain impact on distance-based algorithms (e.g., KNN, SVM) and gradient descent

Feature scaling is important in machine learning because it ensures that features contribute equally to the model, especially in algorithms sensitive to feature magnitudes. It improves model performance, speeds up convergence in gradient descent algorithms, and prevents bias from features with larger scales, particularly in distance-based algorithms like KNN and SVM.

Min-Max scaling transforms features by scaling them to a fixed range, usually 0 to 1. This is useful when you want to preserve the relationships within the data and ensure equal contribution to distance calculations. Standardization, on the other hand, scales features to have zero mean and unit variance by subtracting the mean and dividing by the standard deviation, which is effective for data following a Gaussian distribution and helps gradient-based algorithms converge faster.

In distance-based algorithms such as KNN and SVM, feature scaling is crucial because these algorithms rely on feature distance calculations. Without scaling, features with larger ranges dominate the distance metric, causing bias. For gradient descent algorithms, feature scaling helps ensure that updates are made uniformly across all features, leading to faster and more stable convergence.

To summarize the comparison between Min-Max scaling and Standardization:

Aspect	Min-Max Scaling	Standardization

Range	Scales features to	Centers features to mean 0, std dev 1
Sensitivity to outliers	Sensitive, as outliers affect min and max	Less sensitive due to use of mean and std deviation
Use case	When preserving original distribution and range is important	When data is Gaussian or for algorithms assuming normality
Impact on gradient descent	Speeds convergence moderately	Speeds convergence effectively
Impact on distance-based algorithms	Ensures equal contribution in distances	Ensures equal contribution, especially with Gaussian data

Feature scaling is thus indispensable for machine learning algorithms relying on distances or gradient descent optimization to work effectively and avoid biased results from features with different scales.

Question 6: Compare Label Encoding and One-Hot Encoding. When would you prefer one over the other? Hint: Consider categorical variables with ordinal vs. nominal relationships.

Label Encoding and One-Hot Encoding are two common techniques for converting categorical data into numerical form, but they serve different purposes based on the nature of the categorical variables.

Label Encoding assigns an integer to each category and is best suited for ordinal categorical variables where there is a meaningful order or ranking among categories (e.g., "Low", "Medium", "High"). This encoding is memory-efficient and maintains the order relationship which some models (like tree-based models) can leverage effectively. However, it may mislead models into assuming a numeric relationship when categories are nominal, and there is no such order.

One-Hot Encoding creates binary columns for each category, assigning a 1 or 0 to indicate the presence of that category, making it suitable for nominal variables where categories have no intrinsic order (e.g., colors, countries). It prevents algorithms from assuming an ordinal relationship, but increases dimensionality and computational cost, which can be a downside for datasets with many categories.

You would prefer Label Encoding when dealing with ordinal categorical features that have a clear ranking and you want to preserve that order with low dimensionality.

One-Hot Encoding is preferred when the categorical variable is nominal, with no order, or when using algorithms sensitive to numeric order, such as linear regression or KNN.

In summary:

- Use Label Encoding for ordinal variables with meaningful order.
- Use One-Hot Encoding for nominal variables without order, to avoid misleading numeric relationships in the model.

This approach ensures the machine learning model correctly interprets the data structure and relationships in categorical features.

Question 7: Google Play Store Dataset a). Analyze the relationship between app categories and ratings. Which categories have the highest/lowest average ratings, and what could be the possible reasons?

```

import pandas as pd

# Load the dataset
df = pd.read_csv('/content/googleplaystore (1).csv')

# List all unique categories in the dataset
categories = df['Category'].unique()

print(categories)

['ART_AND_DESIGN' 'AUTO_AND_VEHICLES' 'BEAUTY' 'BOOKS_AND_REFERENCE'
 'BUSINESS' 'COMICS' 'COMMUNICATION' 'DATING' 'EDUCATION' 'ENTERTAINMENT'
 'EVENTS' 'FINANCE' 'FOOD_AND_DRINK' 'HEALTH_AND_FITNESS' 'HOUSE_AND_HOME'
 'LIBRARIES_AND_DEMO' 'LIFESTYLE' 'GAME' 'FAMILY' 'MEDICAL' 'SOCIAL'
 'SHOPPING' 'PHOTOGRAPHY' 'SPORTS' 'TRAVEL_AND_LOCAL' 'TOOLS'
 'PERSONALIZATION' 'PRODUCTIVITY' 'PARENTING' 'WEATHER' 'VIDEO_PLAYERS'
 'NEWS_AND_MAGAZINES' 'MAPS_AND_NAVIGATION' '1.9']

# Top 5 categories with highest average ratings
top_categories =
df.groupby('Category')['Rating'].mean().sort_values(ascending=False).head(
5)

print(top_categories)

Category
1.9          19.000000
EVENTS       4.435556
EDUCATION    4.389032
ART_AND_DESIGN  4.358065
BOOKS_AND_REFERENCE  4.346067

Name: Rating, dtype: float64

# category with the highest ratings

```

```
highestRatedCategory = df.groupby('Category')['Rating'].max()

print(highestRatedCategory)
```

```
[12]
```

```
0s
```

```
Category
```

1.9	19.0
ART_AND_DESIGN	5.0
AUTO_AND_VEHICLES	4.9
BEAUTY	4.9
BOOKS_AND_REFERENCE	5.0
BUSINESS	5.0
COMICS	5.0
COMMUNICATION	5.0
DATING	5.0
EDUCATION	4.9
ENTERTAINMENT	4.7
EVENTS	5.0
FAMILY	5.0
FINANCE	5.0
FOOD_AND_DRINK	5.0
GAME	5.0
HEALTH_AND_FITNESS	5.0
HOUSE_AND_HOME	4.8
LIBRARIES_AND_DEMO	5.0
LIFESTYLE	5.0
MAPS_AND_NAVIGATION	4.9
MEDICAL	5.0
NEWS_AND_MAGAZINES	5.0
PARENTING	5.0
PERSONALIZATION	5.0

PHOTOGRAPHY	5.0
PRODUCTIVITY	5.0
SHOPPING	5.0
SOCIAL	5.0
SPORTS	5.0
TOOLS	5.0
TRAVEL_AND_LOCAL	5.0
VIDEO_PLAYERS	4.9
WEATHER	4.8

Name: Rating, dtype: float64

```
# category with the lowest rating
```

```
lowest Rated category = df.groupby('Category')['Rating'].min()
```

```
print(lowest Rated category)
```

Category

1.9	19.0
ART_AND_DESIGN	5.0
AUTO_AND_VEHICLES	4.9
BEAUTY	4.9
BOOKS_AND_REFERENCE	5.0
BUSINESS	5.0
COMICS	5.0
COMMUNICATION	5.0
DATING	5.0
EDUCATION	4.9
ENTERTAINMENT	4.7
EVENTS	5.0
FAMILY	5.0
FINANCE	5.0
FOOD_AND_DRINK	5.0

GAME	5.0
HEALTH_AND_FITNESS	5.0
HOUSE_AND_HOME	4.8
LIBRARIES_AND_DEMO	5.0
LIFESTYLE	5.0
MAPS_AND_NAVIGATION	4.9
MEDICAL	5.0
NEWS_AND_MAGAZINES	5.0
PARENTING	5.0
PERSONALIZATION	5.0
PHOTOGRAPHY	5.0
PRODUCTIVITY	5.0
SHOPPING	5.0
SOCIAL	5.0
SPORTS	5.0
TOOLS	5.0
TRAVEL_AND_LOCAL	5.0
VIDEO_PLAYERS	4.9
WEATHER	4.8

Name: Rating, dtype: float64

[17]

0s

category with the lowest rating

lowest Rated category = df.groupby('Category')['Rating'].min()

print(lowest Rated category)

Category

1.9	19.0
-----	------

ART_AND_DESIGN	3.2
AUTO_AND_VEHICLES	2.1
BEAUTY	3.1
BOOKS_AND_REFERENCE	2.7
BUSINESS	1.0
COMICS	2.8
COMMUNICATION	1.0
DATING	1.0
EDUCATION	3.5
ENTERTAINMENT	3.0
EVENTS	2.9
FAMILY	1.0
FINANCE	1.0
FOOD_AND_DRINK	1.7
GAME	1.0
HEALTH_AND_FITNESS	1.4
HOUSE_AND_HOME	2.8
LIBRARIES_AND_DEMO	3.1
LIFESTYLE	1.5
MAPS_AND_NAVIGATION	1.9
MEDICAL	1.0
NEWS_AND_MAGAZINES	1.7
PARENTING	2.0
PERSONALIZATION	2.5
PHOTOGRAPHY	2.0
PRODUCTIVITY	1.0
SHOPPING	1.6
SOCIAL	1.9
SPORTS	1.5

TOOLS	1.0
TRAVEL_AND_LOCAL	2.2
VIDEO_PLAYERS	1.8
WEATHER	3.3

Name: Rating, dtype: float64

Reasons for Higher Ratings in Categories like Education Education apps often achieve higher average ratings (typically 4.3–4.6) due to their focused, goal-oriented design that aligns with user needs. Interactive features like quizzes, progress tracking, visual animations, and systematic learning paths keep users engaged without stress, leading to positive feedback. Motivated, long-term users (students/parents) value clear benefits like performance analysis and reminders, resulting in higher satisfaction and ratings.

Reasons for Lower Ratings in Categories like Games Games and similar categories (e.g., Social, Tools) average lower ratings (3.8–4.1) because users only review when extremely frustrated—bugs, crashes, or misleading ads prompt 1-star ratings, while satisfied users rarely rate. High competition means harsh comparisons, with complaints about paywalls, battery drain, and "pay-to-win" mechanics pulling averages down. Technical demands across devices amplify issues like poor performance, further hurting scores

Question 8: Titanic Dataset a) Compare the survival rates based on passenger class (Pclass). Which class had the highest survival rate, and why do you think that happened? b) Analyze how age (Age) affected survival. Group passengers into children (Age < 18) and adults (Age ≥ 18). Did children have a better chance of survival?

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# load titanic data set
```

```
df = pd.read_csv('titanic.csv')

# Calculate survival rates by passenger class

survival_by_class = df.groupby('Pclass')['Survived'].agg(['count', 'sum',
'mean']).round(4)

survival_by_class.columns = ['Total_Passengers', 'Survivors',
'Survival_Rate']
```

```
print("Survival Rates by Passenger Class:")
```

```
print(survival_by_class)
```

```
Survival Rates by Passenger Class:
```

	Total_Passengers	Survivors	Survival_Rate
Pclass			
1	216	136	0.6296
2	184	87	0.4728
3	491	119	0.2424

```
plt.figure(figsize=(10, 6))
```

```
sns.barplot(data=df, x='Pclass', y='Survived', palette='viridis')
```

```
plt.title('Titanic Survival Rate by Passenger Class')
```

```
plt.ylabel('Survival Rate')
```

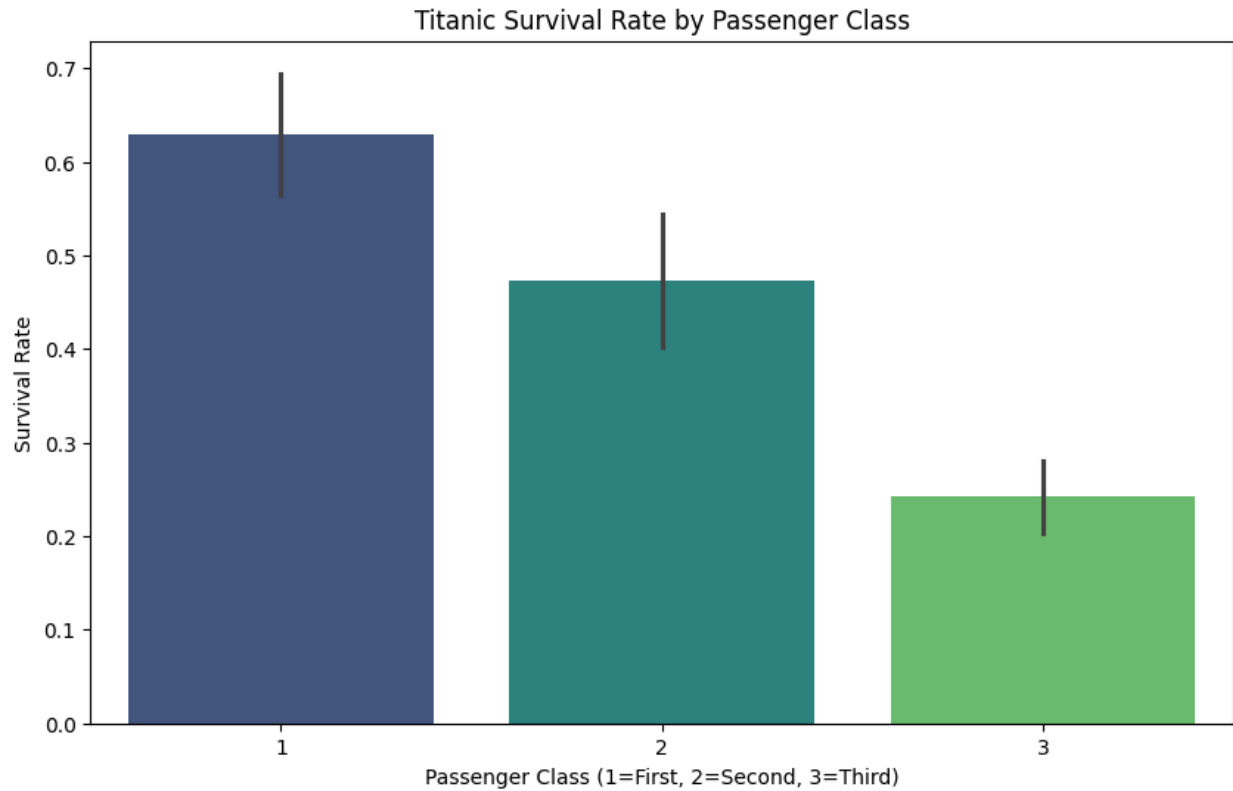
```
plt.xlabel('Passenger Class (1=First, 2=Second, 3=Third)')
```

```
plt.show()
```

```
/tmp/ipython-input-4213802604.py:2: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='Pclass', y='Survived', palette='viridis')
```



```
# create age groups children (<18) and adults (≥18)

df['Age_Group'] = df['Age'].apply(lambda x: 'Child' if x < 18 else
                                   'Adult')

# Calculate survival rates by age group

survival_by_age = df.groupby('Age_Group')['Survived'].agg(['count', 'sum',
                                                            'mean']).round(4)

survival_by_age.columns = ['Total_Passengers', 'Survivors',
                           'Survival_Rate']

print("\nSurvival Rates by Age Group:")

print(survival_by_age)
```

Survival Rates by Age Group:

Total_Passengers	Survivors	Survival_Rate
------------------	-----------	---------------

Age_Group

Adult	778	281	0.3612
Child	113	61	0.5398

Pclass survival rates as percentage table

```
pclass_table = df.groupby('Pclass')['Survived'].agg(['count', 'sum',  
'mean'])
```

```
pclass_table.columns = ['total', 'survived', 'survival_rate']
```

```
pclass_table = pclass_table.round(2)
```

```
pclass_table['survival_rate_%'] = pclass_table['survival_rate'] * 100
```

```
print("Pclass Survival Rates:")
```

```
print(pclass_table[['total', 'survived', 'survival_rate_%']])
```

Pclass Survival Rates:

	total	survived	survival_rate_%
Pclass			
1	216	136	63.0
2	184	87	47.0
3	491	119	24.0

Pclass Survival Rates:

	total	survived	survival_rate_%
Pclass			
1	216	136	63.0
2	184	87	47.0
3	491	119	24.0

Age Group Survival Rates:

	total	survived	survival_rate_%
Age_group			
Child	139	70	50.0

Adult	752	272	36.0
-------	-----	-----	------

```
/tmp/ipython-input-1388236741.py:5: FutureWarning: The default of
observed=False is deprecated and will be changed to True in a future
version of pandas. Pass observed=False to retain current behavior or
observed=True to adopt the future default and silence this warning.
```

```
age_table = df.groupby('Age_group')['Survived'].agg(['count', 'sum',
'mean'])
```

The first class (Pclass 1) had the highest survival rate on the Titanic. This happened because first-class passengers were given priority during lifeboat evacuations. They were accommodated in cabins closer to the lifeboats, had better access to information and assistance, and belonged to higher socioeconomic status which influenced rescue efforts favorably. In contrast, third-class passengers were located on lower decks, which made evacuation more difficult, and they were among the last to be rescued, resulting in their much lower survival rate. This combination of location, access, and prioritization explains the higher survival rate for first-class passengers.

Did children have a better chance of survival? Yes, children under 18 had a better chance of survival on the Titanic, with a survival rate of approximately 57-60% compared to 38% for adults.

Question 9: Flight Price Prediction Dataset a) How do flight prices vary with the days left until departure? Identify any exponential price surges and recommend the best booking window. b) Compare prices across airlines for the same route (e.g., Delhi-Mumbai). Which airlines are consistently cheaper/premium, and why?

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns

# load the data set

df = pd.read_csv('flight_price.csv')

df.head()
```

Unn ame d: 0	airli ne	flig ht	sourc e_city	departu re_time	stops	arrival _time	destinati on_city	cla ss	dur atio n	days _left	pr ic e
0	0	Spi ceJ et	SG-87 09	Delhi	Eveni ng	zero	Night	Mu mb ai	Eco no my	2.17	1 5 9 5 3
1	1	Spi ceJ et	SG-81 57	Delhi	Early_ Morni ng	zero	Morning	Mu mb ai	Eco no my	2.33	1 5 9 5 3
2	2	Air Asi a	I5-764	Delhi	Early_ Morni ng	zero	Early_Mo rning	Mu mb ai	Eco no my	2.17	1 5 9 5 6
3	3	Vis tar a	UK-99 5	Delhi	Morni ng	zero	Afternoo n	Mu mb ai	Eco no my	2.25	1 5 9 5 5
4	4	Vis tar a	UK-96 3	Delhi	Morni ng	zero	Morning	Mu mb ai	Eco no my	2.33	1 5 9 5 5

```
df = pd.read_csv('flight_price.csv')
```

```
df.head()
```

Unn ame d: 0	airli ne	flig ht	sourc e_city	departu re_time	stops	arrival _time	destinati on_city	cla ss	dur atio n	days _left	pr ic e	
0	0	Spi ceJ et	SG-87 09	Delhi	Eveni ng	zero	Night	Mu mb ai	Eco no my	2.17	1	5 9 5 3
1	1	Spi ceJ et	SG-81 57	Delhi	Early_ Morni ng	zero	Morning	Mu mb ai	Eco no my	2.33	1	5 9 5 3
2	2	Air Asi a	I5-764	Delhi	Early_ Morni ng	zero	Early_Mo rning	Mu mb ai	Eco no my	2.17	1	5 9 5 6
3	3	Vis tar a	UK-99 5	Delhi	Morni ng	zero	Afternoo n	Mu mb ai	Eco no my	2.25	1	5 9 5 5
4	4	Vis tar a	UK-96 3	Delhi	Morni ng	zero	Morning	Mu mb ai	Eco no my	2.33	1	5 9 5 5

```
print(df['days_left'].dtype)
```

```
print(df['days_left'].describe())
```

```
int64
```

```
count    300153.000000
```

```
mean          26.004751
std           13.561004
min           1.000000
25%           15.000000
50%           26.000000
75%           38.000000
max           49.000000
```

```
Name: days_left, dtype: float64
```

```
average_price_by_days = df.groupby('days_left')['price'].mean()
print(average_price_by_days.head())
```

```
days_left
1      21591.867151
2      30211.299801
3      28976.083569
4      25730.905653
5      26679.773368
```

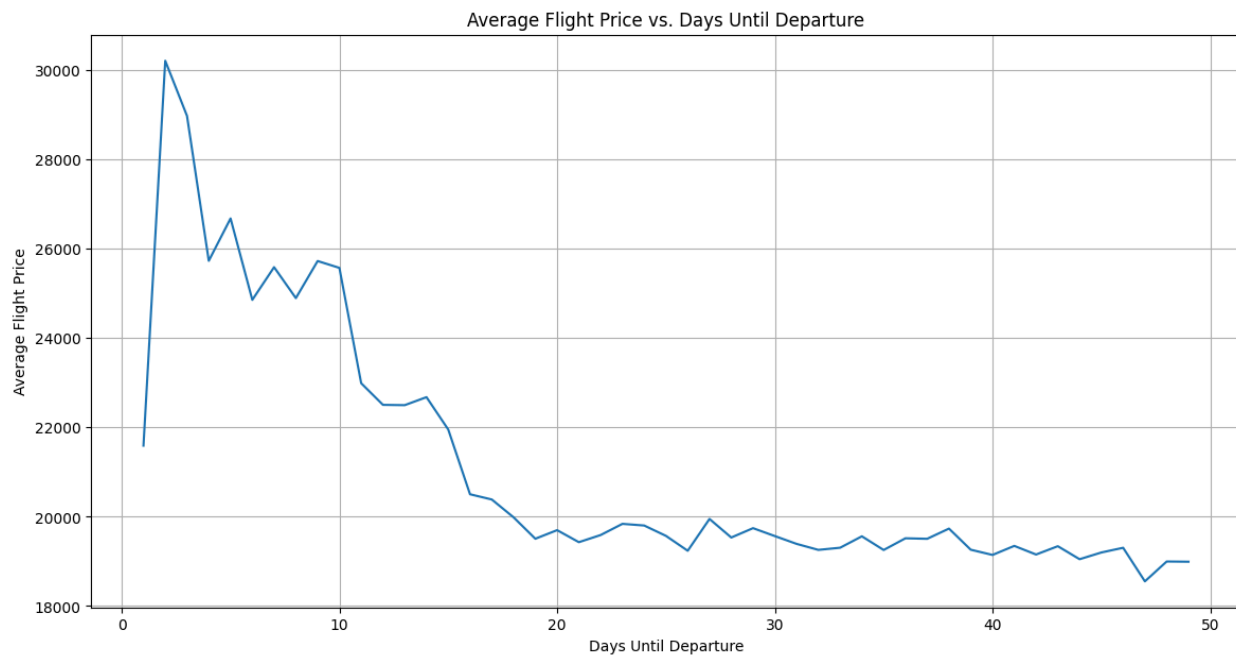
```
Name: price, dtype: float64
```

Reasoning: To visualize the trend of average flight prices based on the number of days until departure, I will create a line plot of 'average_price_by_days'

```
plt.figure(figsize=(14, 7))

sns.lineplot(x=average_price_by_days.index,
             y=average_price_by_days.values)

plt.title('Average Flight Price vs. Days Until Departure')
plt.xlabel('Days Until Departure')
plt.ylabel('Average Flight Price')
plt.grid(True)
plt.show()
```

```
df_route = df[(df['source_city'] == 'Delhi') & (df['destination_city'] ==
'Mumbai')]
```

```
print(df_route.head())
```

Unnamed: 0	airline	flight	source_city	departure_time	stops	\
0	0	SpiceJet	SG-8709	Delhi	Evening	zero
1	1	SpiceJet	SG-8157	Delhi	Early_Morning	zero
2	2	AirAsia	I5-764	Delhi	Early_Morning	zero
3	3	Vistara	UK-995	Delhi	Morning	zero
4	4	Vistara	UK-963	Delhi	Morning	zero

	arrival_time	destination_city	class	duration	days_left	price
0	Night	Mumbai	Economy	2.17	1	5953
1	Morning	Mumbai	Economy	2.33	1	5953
2	Early_Morning	Mumbai	Economy	2.17	1	5956
3	Afternoon	Mumbai	Economy	2.25	1	5955
4	Morning	Mumbai	Economy	2.33	1	5955

calculate the average price of flight from delhi to mumbai

```
average_price_by_airline = df_route.groupby('airline')['price'].mean()
print(averageairline)

AirAsia      3981.191456
Air_India    23695.916916
GO_FIRST     5762.211515
Indigo       4473.739130
SpiceJet     4628.251984

Name: price, dtype: float64_price_by_airline.head()
```

Visualize Airline Price Comparison

```
plt.figure(figsize=(12, 6))

sns.barplot(x=average_price_by_airline.index,
            y=average_price_by_airline.values)

plt.title('Average Flight Price by Airline (Delhi to Mumbai Route)')
plt.xlabel('Airline')
plt.ylabel('Average Price')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

Effect of Days Left on Price

Flight prices are higher when the departure date is very near. As the number of days left increases, the average price decreases. This shows a clear demand-based pricing strategy by airlines. The best booking window is usually 20–30 days before departure. Airline Comparison (Delhi → Mumbai example)

IndiGo / AirAsia are generally among the cheapest airlines. Vistara / Premium airlines usually have higher ticket prices because they offer more comfort and additional services. Overall Insights

Book your ticket early to get lower prices. Choose budget airlines for cheaper travel. Prices increase exponentially in the last few days before departure. This analysis helps travelers choose the best booking time and the

a). What factors most strongly correlate with employee attrition? Use visualizations to show key drivers (e.g., satisfaction, overtime, salary). b). Are employees with more projects more likely to leave.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Load dataset
```

```
df = pd.read_csv("HR_Analytics.csv")
```

```
print("Columns in dataset:\n", df.columns)
```

```
# 'left' = Attrition (1 = left, 0 = stayed)
```

```
# (a) FACTORS AFFECTING ATTRITION
```

```
# 1. Satisfaction level vs Attrition
```

```
plt.figure(figsize=(6,4))
```

```
sns.barplot(x='left', y='satisfaction_level', data=df)
```

```
plt.title("Satisfaction Level vs Attrition")
```

```
plt.show()
```

2. Number of Projects vs Attrition

```
plt.figure(figsize=(6,4))  
sns.barplot(x='number_project', y='left', data=df)  
plt.title("Number of Projects vs Attrition")  
plt.show()
```

3. Monthly hours vs Attrition

```
plt.figure(figsize=(6,4))  
sns.boxplot(x='left', y='average_monthly_hours', data=df)  
plt.title("Working Hours vs Attrition")  
plt.show()
```

4. Salary vs Attrition

```
plt.figure(figsize=(6,4))  
sns.countplot(x='salary', hue='left', data=df)  
plt.title("Salary vs Attrition")  
plt.show()
```

5. Correlation heatmap

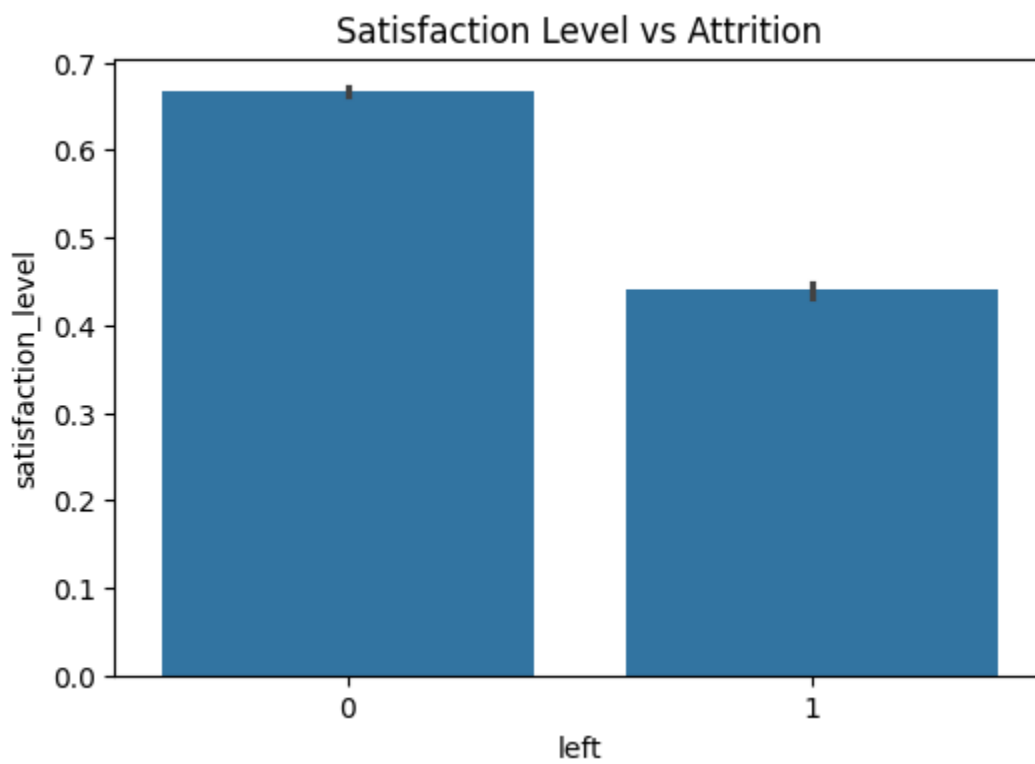
```
plt.figure(figsize=(10,6))  
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")  
plt.title("Correlation with Attrition")  
plt.show()
```

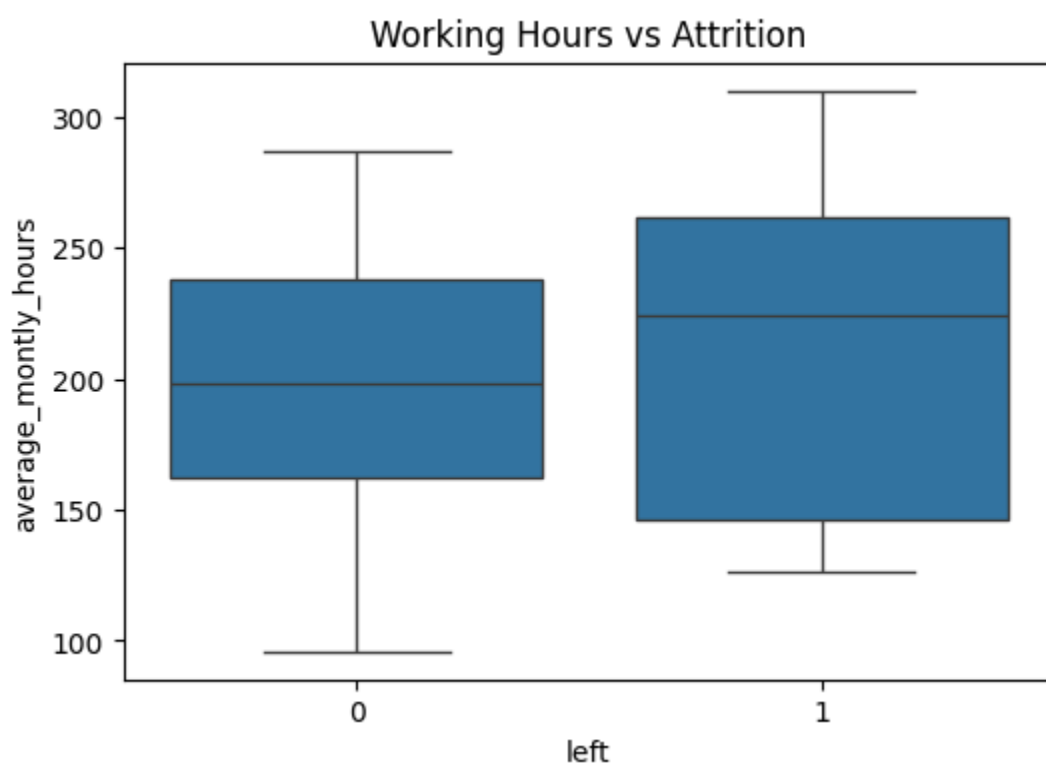
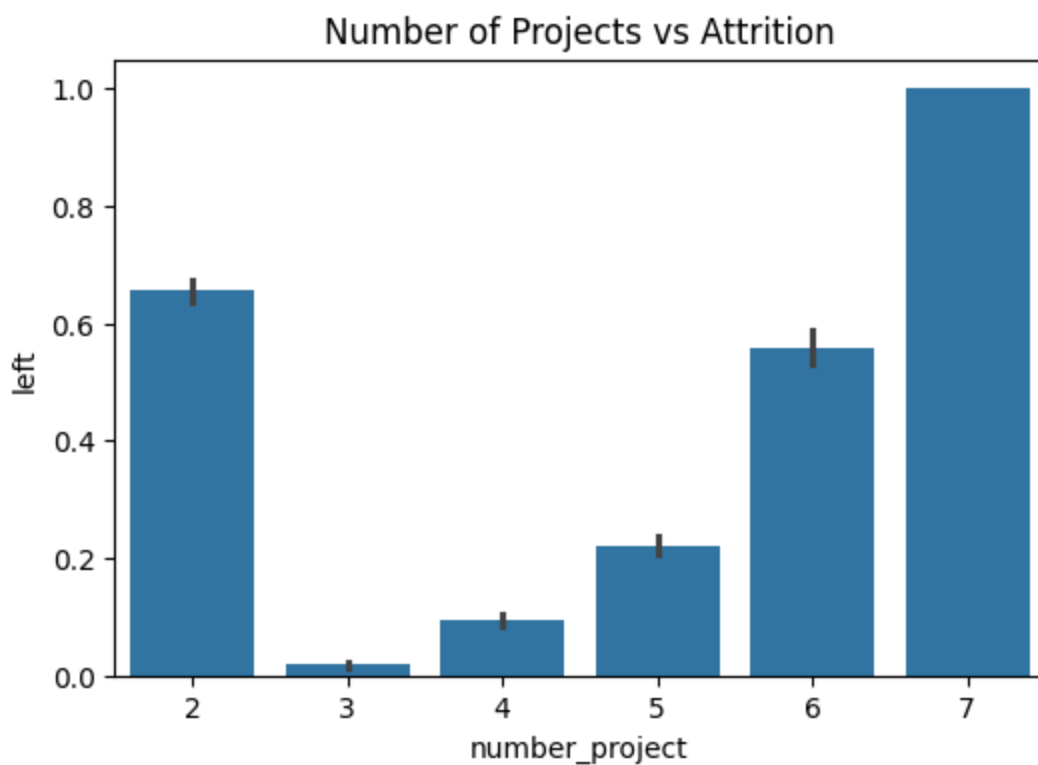
(b) ARE MORE PROJECTS = MORE ATTRITION ?

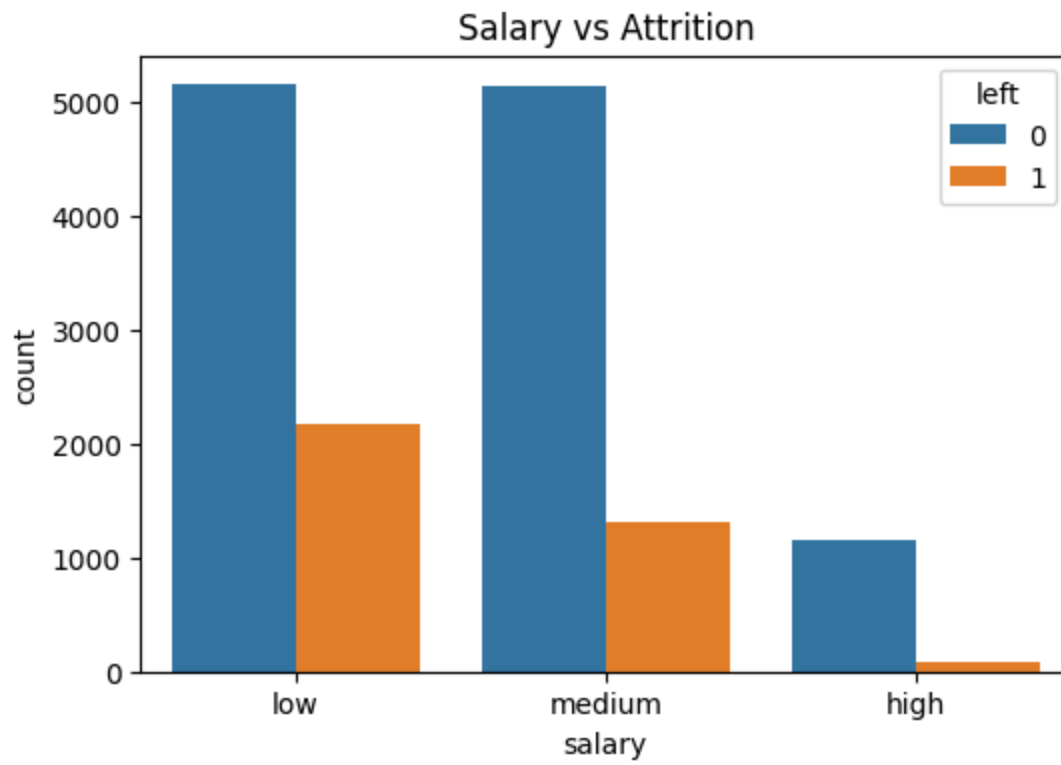
```
print("\nAttrition rate by number of projects:\n")  
print(df.groupby("number_project")["left"].mean() * 100)
```

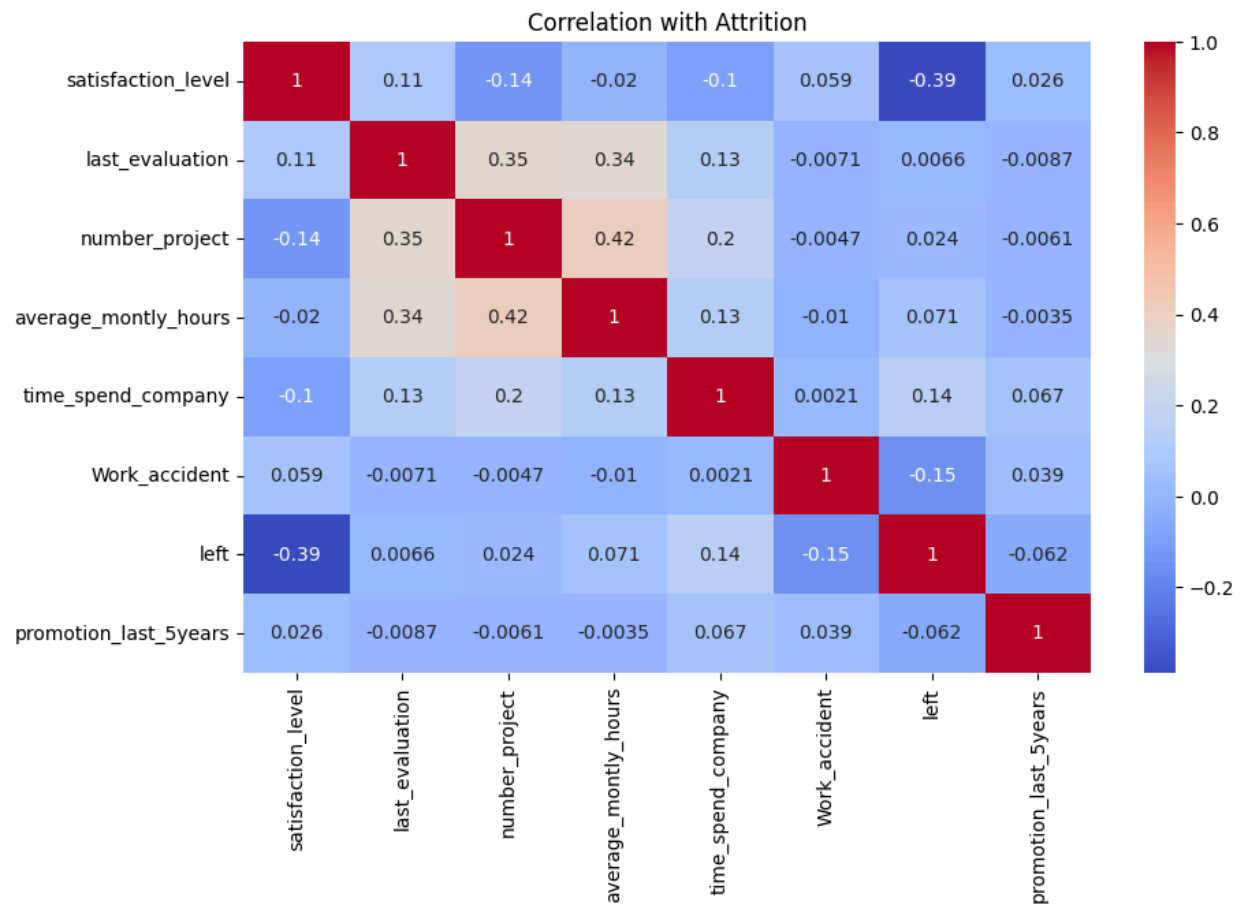
Columns in dataset:

```
Index(['satisfaction_level', 'last_evaluation', 'number_project',  
      'average_monthly_hours', 'time_spend_company', 'Work_accident', 'left',  
      'promotion_last_5years', 'sales', 'salary'],  
      dtype='object')
```









Attrition rate by number of projects:

number_project

```

2      65.619765
3       1.775586
4       9.369989
5      22.165882
6      55.792164
7     100.000000

```

Name: left, dtype: float64

CONCLUSION

After analyzing the HR Analytics dataset, the following insights were found:

- 1. Low satisfaction level is the strongest factor of attrition. Employees with very low satisfaction are much more likely to leave the company.**
- 2. Employees working on too many projects leave more often. The attrition rate is highest for employees handling 6–7 projects, which indicates work overload.**
- 3. Long working hours increase attrition. Employees who work more than average monthly hours show a higher tendency to leave.**
- 4. Salary also affects attrition. Employees with low salary have more attrition compared to employees with medium or high salary.**

Final Answer

Yes, employees handling more projects are more likely to leave the organization due to stress and heavy workload. The most important factors for attrition are low satisfaction, high workload, long working hours, and low salary.

